

Optimizing the Concurrent Multiversion B-Tree (cMVBT): A Bottom-Up Tour of the System, its Tricks, and its OSIC Integration

Amir Tonta

August 2026

Abstract

This document collects, in one place, the optimizations applied to the cMVBT-OSIC implementation over roughly two and a half years and ~280 commits. It is organized bottom-up: we start at the physical layout of a page on disk/in memory, work our way up through concurrency control, structural maintenance (splits/merges), garbage collection, and root indexing, and only then reach the transaction layer, where the centerpiece optimization of this document lives — the *self-overwrite* trick for repeated updates to the same key within one transaction. We then step back and look at two cross-cutting, system-wide themes: *sharding* (partitioning shared, contended bookkeeping state) and the integration of *OSIC* (Ordered Snapshot Instant Commit), the durability/visibility model from LeanStore’s snapshot-isolation paper. A running theme throughout is a recurring engineering pattern: early designs patched races with atomics and memory fences; later designs instead restructured the data so the race became structurally impossible.

Contents

1	Introduction and roadmap	1
2	Physical layer: pages and multiversion records	2
3	Concurrency control: from lock coupling to optimistic lock coupling	3
3.1	The protocols that were tried	3
3.2	The rise and fall of ORWC	3
3.3	The SmartCell latch	4
3.4	Chasing fences out of the hot path	4
4	Structural maintenance: splits, merges, and page capacity	6
4.1	Capacity tuning	6
4.2	Split: choosing where to cut	6
4.3	Merge: choosing what to combine	6
4.4	A week of hardening (2026-07-31 – 08-02)	7
5	The self-overwrite optimization	7
5.1	The problem	7
5.2	The fix	7
5.3	The chronological story	8
5.4	What remains open	9
6	Garbage collection: reclaiming dead blocks without contention	9

6.1	From a global mutex to sharded, lock-free reclaim	9
6.2	Two more races found by ThreadSanitizer, and how each was closed structurally	10
7	Root indexing strategies	11
8	Sharding as a system-wide pattern	11
8.1	Tables as shards: one tree per table	11
8.2	GC dead-block tracker: own-shard-first, round-robin steal	12
8.3	TxContext: cache-padded, per-worker atomic slots	12
8.4	Thread-local worker identity and snapshot cache	13
9	OSIC integration: instant commit, lazily hardened	13
9.1	What OSIC is	13
9.2	How cMVBT's commit path works today	13
9.3	Highlights and differences versus the paper	14
9.4	Evolution: two eras	15
9.5	Known, documented limitations	16
10	Timeline	16
11	Conclusion	17

1 Introduction and roadmap

cMVBT is a concurrent, in-memory, multiversion B-tree that implements the asymptotically optimal MVBT structure of Becker et al. [1], extended with a full multiversion concurrency control (MVCC) protocol described in [2]. On top of the tree itself, the system integrates *Ordered Snapshot Instant Commit* (OSIC), the durability/visibility scheme from LeanStore's scalable snapshot-isolation work [3], so that transaction commits become visible instantly (in commit order) while the write-ahead log (WAL) is hardened to disk lazily, in batches, out of the commit's critical path.

Reading a codebase's git history commit-by-commit gives you *what changed*; it rarely gives you *why*, and it never gives you the shape of the whole system. This document reconstructs both: for every subsystem below we explain the current design, then the sequence of failed/superseded designs that preceded it and the concrete bug or profiling number that motivated each change.

How to read this document. It is ordered from the ground up, each section building on the one before it:

1. **Physical layer** (§2): how a page stores multiple versions of a key.
2. **Concurrency control** (§3): how threads latch pages built from §2 without blocking each other unnecessarily.
3. **Structural maintenance** (§4): how pages from §2, latched per §3, split and merge as they fill up or empty out.
4. **The self-overwrite trick** (§5): the centerpiece optimization, sitting squarely on top of §2 and §4 — how the transaction layer avoids ever handing the structural-maintenance code in §4 a page it cannot split or merge.
5. **Garbage collection** (§6): how pages retired by §4 are reclaimed once no live snapshot needs them.

6. **Root indexing** (§7): how a reader finds the correct root for its snapshot in the first place.
7. **Sharding** (§8): a cross-cutting pattern applied to several of the subsystems above (GC, per-worker bookkeeping, and the database’s own table layout) to remove contention on shared state.
8. **OSIC integration** (§9): the transaction-commit and durability layer sitting on top of everything else, and how it differs from (and reuses) the ideas in [3].
9. **Timeline** (§10): a chronological index of the commits discussed, for cross-reference.

2 Physical layer: pages and multiversion records

Every leaf page (`src/mv_page_model/leaf_page.rs`) is a fixed-capacity, *append-only* array:

```
record_data: [MaybeUninit<RecordPoint<Key, Payload>>; NUM_RECORDS]
```

plus a packed `len` counter tracking active vs. dead slots (`leaf_page.rs:145-147`). `push_uncommitted` (`leaf_page.rs:163-180`) only ever writes at the next free slot and asserts `index < NUM_RECORDS`. Crucially, **there is no in-place removal**: an obsolete, aborted, or deleted entry is only ever *marked* dead/invalid via `VersionInfo::delete/invalidate` (`src/mv_record_model/version_info.rs:147-164`); it is never physically dropped until the next structural-maintenance operation (§4) rebuilds the page from scratch.

This means a single key can accumulate a *physical chain* of several `RecordPoint` tuples in one page — one per insert/update/delete against that key — found by scanning backward (`rfind`) for the most recent live entry (`leaf_page.rs:335-337`). Each tuple carries a `VersionInfo` (`version_info.rs:37-40`): an `insertion_stamp: TxStamp` (a packed `(worker\_id, ts\_start)`, constant for an entire transaction — not per write, see §5) and an optional `deletion_stamp`. This packing is what later makes “was this the newest version, and did I, this very transaction, write it” a single equality check rather than a heuristic.

Structural maintenance (§4) decides what to keep during a rebuild via `record_survives_gc` (`src/mv_tree/smo.rs:736-744`): a record survives if it is live, *or* if it was deleted by a transaction whose snapshot some other, still-live transaction may still need (`ctx.is_snapshot_live`). An *uncommitted* record’s fate is undetermined until commit or abort: it can be neither discarded (might still be committed) nor coalesced with anything (nothing superseded it yet). This single fact — that live, dead-but-still-needed, and uncommitted entries all occupy physical slots indefinitely — is the root cause behind the two big problems solved in §4–§5: pages that cannot be split cleanly, and pages that fill up with garbage from a single busy transaction.

The internal (index) page format underwent a parallel simplification, discussed together with garbage collection in §6: an early design tracked obsolete child pointers with an atomic “obsolete mark” bit mutated in place on an already-published, concurrently-read slot; the current design (`fe03d90`) instead derives liveness from the fact that live sibling key intervals are always pairwise disjoint, so no slot is ever mutated after publication (`InternalPage::live_mask()`).

3 Concurrency control: from lock coupling to optimistic lock coupling

3.1 The protocols that were tried

The lock-mode taxonomy predates the multiversion (`mv_-prefixed`) rewrite and still lives in a sister crate, `CCBPlusTree`, whose `LockingStrategy` enum defines six variants:

- **MonoWriter** — a single writer, unlimited readers, no real concurrency control at all.
- **LockCoupling (LC)** — classic hand-over-hand exclusive latching down the tree.
- **ORWC** — “Optimistic Read-Write Coupling”: readers are free; writers use a level-bounded, optimistic-then-blocking coupling scheme with an “optimistic upgrade” path.
- **OLC** — Optimistic Lock Coupling (Leis-style): readers are fully lock-free, relying only on version validation; writers use a versioned, seqlock-like latch.
- **LHL** (LightweightHybridLock) — level-bounded: optimistic below a threshold depth, exclusive above it.
- **HL** (HybridLocking) — bounded-retry hybrid read/write locking.

Only **OLC** survives in the current codebase: the write/read fast paths (`src/mv_query/olc_query.rs`, `src/mv_query/query.rs`, `src/mv_query/rand_query.rs`) are hard-wired to OLC-style traversal (`traversal_write_olc`, `traverse_read_key`, `try_retire`, `upgrade_write_lock`). MonoWriter, LockCoupling, LHL and HL were prototyped in early 2024 (`dcf3b2b`) but never carried into the multiversion rewrite.

3.2 The rise and fall of ORWC

ORWC is worth its own subsection because it is the one protocol that was implemented, iterated on for roughly a year, and then deliberately removed — a useful cautionary tale about optimistic upgrade paths.

- `d565b2d` (2024-01-22) — first sign of trouble: *“ORWC seem to have an issue.”*
- `ac006fe` (2024-01-26) — the root is made upgradeable optimistically “by all protocols — deadlock-free, including for ORWC”, adding a post-upgrade re-validation (`root_guard.deref().unwrap().len()==curr_len`) because a naive before/after version compare cannot detect a writer already mid-flight at both sample points.
- `48c4ded` (2024-05-06) — *“Updated OLC for actual required validations. Fixed ORWC.”* Redundant `fence(Acquire)/fence(SeqCst)` calls are stripped from the traversal, since OLC’s own atomic load/CAS already carries the needed ordering.
- `b1e7092` (2024-05-07) — *“Further fixed ORWC issue with upgrades and free readers sometimes encountering segfaults.”* Guard variants are reworked from holding raw pointers to holding a cloned `SmartCell` handle, because a raw pointer captured at borrow time could dangle across a concurrent reuse.
- `f368155` (2024-05-07, same day) — *“Make standard ORWC mut borrow blocking and only the upgrade non-blocking.”* ORWC’s mutable-borrow path had to give up non-blocking semantics entirely to stay correct; only the optimistic *upgrade* attempt remained non-blocking.
- `3c6da7b` (2024-12-06) — *“Deprecated ORWC.”* The README changes from “ORWC: Unlimited all.” to “ORWC: Deprecated; Optimistic Upgrade not working.”
- `5c26abb` (2024-12-08) — *“Purged ORWC → duplicate with OLC. Cleaned code accordingly.”* ~460 lines removed.

Root cause, in one sentence: ORWC’s optimistic upgrade path never reached a provably correct state — four separate fix attempts over a year (segfaults on free readers, upgrade races, blocking/non-blocking inconsistency) — and once OLC was fully validated it gave the identical guarantee through one simpler seqlock mechanism, making ORWC a pure maintenance liability rather than a genuinely distinct protocol worth keeping.

3.3 The SmartCell latch

`src/mv_sync/smart_cell.rs` implements the versioned latch (“SmartCell”) that every node uses:

- **Layout:** `OptCell<E>{cell:SafeCell<E>,cell_version:AtomicVersion}` — a single 64-bit versioned word per node, with three logical bits packed into its top nibble: a `WRITE_FLAG_VERSION` (seqlock “locked” bit), pin-related flags, and a *permanent* `RETIRED_FLAG_VERSION` tombstone bit.
- **Optimistic read** (`borrow_read`): one `Acquire` load of `cell_version`, masking out the write/retired bits — no lock taken.
- **Upgrade to exclusive** (`upgrade_write_lock`): a CAS from the captured read version to `WRITE_FLAG_VERSION|read_version`, failing fast if the retired bit is already set (a retired cell can never be write-locked again).
- **Unlock:** stores `(write_version+1)\^{WRITE_FLAG_VERSION|retired_bit}` with `Release`. Reading the current retired bit with a plain `Relaxed` load here is safe (nothing else can touch `cell_version` while the writer guard is alive) and, crucially, preserves a same-critical-section `mark_retired()` call instead of clobbering it (see §6 for the bug this fixes).
- **try_retire:** a single CAS straight from a `Reader` snapshot to `RETIRED_FLAG_VERSION|read_latch`, skipping the intermediate exclusive state — sound only for guards that, once excluded, are unconditionally consumed with no “back out” path.

3.4 Chasing fences out of the hot path

A second, quieter optimization story runs in parallel to the protocol story above: once OLC’s own atomic operations were trusted to carry the necessary ordering, standalone memory fences bolted on earlier (out of caution) were incrementally identified as redundant and removed. Table 1 lists the sequence.

Commit	Date	Change	Rationale
48c4ded	2024-05-06	Removed <code>fence(Acquire)/fence(SeqCst)</code> in the traversal loop	The surrounding OLC load/CAS already provides the ordering.
3c6da7b	2024-12-06	Removed <code>fence(Acquire)</code> paired duplicating <code>len.load(Acquire)</code> in page length accessors	An atomic load’s own ordering already suffices.
a7bb29c	2025-04-01	<i>“Avoid unnecessary CAS failures by moving to acquire/release semantics.”</i> CAS failure ordering <code>Relaxed→Acquire</code> ; unlock stores <code>Relaxed→Release</code>	Establishes the happens-before edges OLC readers actually rely on, cutting spurious CAS retries from under-synchronized reads racing writers.
a518eed	2025-04-25	<i>“Updated atomics, removed fences.”</i> Deleted standalone fences immediately adjacent to an atomic store/load on the same word	Pure dead weight once the adjacent atomic op’s ordering subsumed it.

Commit	Date	Change	Rationale
2a2be93	2026-06-22	Page soft-commit length bumps <code>Release</code> → <code>Relaxed</code>	The subsequent write-unlock’s own <code>Release</code> on the latch already publishes everything written before it in program order.
40b00fd	2026-06-30	<code>Node::m_type AtomicUsize</code> → <code>usize</code> ; removed <code>fence(Acquire)</code> in <code>DropforNode</code>	The type tag is only ever mutated while the caller holds the exclusive write lock; the latch’s own release already publishes it.
45c7c21/fd921026	2026-07-21	“Remove conservative child latch”: <code>try_retire</code> lets read-only, unconditionally-consumed guards skip a full write-lock upgrade during split/merge	Removes one CAS/lock-acquire per split/merge for guards with no backout path.

Table 1: Chronology of fence removal and ordering tightening in the OLC latch path.

Two further, structural memory-layout optimizations sit alongside this fence-removal story:

- 7f2bd09 (2026-05-21) — “*Pin Root* for root_live reorg...Move node tag to single cache-line.*” `Node` gains `\#[repr(C,align(64))]` plus explicit padding so the hot, frequently-pollled page-type tag does not false-share a cache line with the rest of the node’s content; the root index itself moves from a separate `parking_lot` mutex onto the same `SmartCell/SmartGuard` machinery as ordinary blocks.
- e69ed87/75453aa/d15c693/45cb048 (2026-05-26 to 28, “Addressing memory optimizations”) — among other changes, guard lifetimes are changed from `'static` to an explicit `'a` (fixing a `mem::transmute`’d dangling reference for height-0 trees), and, tellingly, 45cb048 *reverts* an experiment that had range-query iterators hold a long-lived `BlockGuard` across the whole iteration: a `'static` lifetime guard held for an iterator’s entire life is exactly the kind of long-lived, never-revalidated read that GC’s reclaim decision does not know about. The revert restores a bare `BlockRef`, re-borrowed fresh on each step, keeping GC’s liveness bookkeeping (§6) sound.

4 Structural maintenance: splits, merges, and page capacity

Structure-modification operations (SMOs) live in `src/mv_tree/smo.rs` and decide, on the cold path, whether a page needs to split (overflow) or merge (underflow), and how.

4.1 Capacity tuning

Two independent tuning passes shrink the gap between “nominally full” and “actually lands cleanly on hardware pages”:

- 34555e2 (2026-06-20) — “*Updated overflow count, was set too eager. Its enough to trigger at ≤ 1 slot left for keys, not ≤ 3 .*” The internal-page overflow trigger (`overflow_keys_count()`) moved from `max_keys()-3` to `max_keys()-1`, so a page is allowed to fill much closer to capacity before an SMO is even attempted.

- **b29d347** (2026-08-02) — “*FAN_OUT/NUM_RECORDS: 125 → 123, so the real allocated block lands exactly on a 4KB page.*” The nominal record count was tuned down by two so that the *actual* heap allocation (`OptCell<Block<...>>`, not `Block` alone — the wrapping cell adds bytes) lands on a 4KB boundary: measured to move page-aligned allocations from 25% to 100% of the time, with a 9–12% reduction in dTLB load misses.

4.2 Split: choosing where to cut

`split()` decides between a **VERSION_SPLIT** (compact obsolete/dead versions out of one page in place) and a **KEY_SPLIT** (cut the page in two at some key boundary). Two real bugs surfaced here under concurrent TPC-C stress in July–August 2026:

- **2a2607e** (2026-08-02) — “*Fix split() off-by-one that let a full-but-protected leaf overflow.*” The `VERSION_SPLIT`-vs-`KEY_SPLIT` decision used a strict `survivor_count > capacity` test, so a page whose survivors landed at *exactly* capacity wrongly took the “just compact” path, producing an already-100%-full result with no room left for the pending write (a panic on a root-leaf tree, or an infinite retry loop otherwise). Fixed by `survivor_count >= capacity` (`smo.rs:1247`). This turned out to be the actual root cause of a stress-test failure originally suspected to be about hot-key version tearing (see §5) — the failing page in fact held several distinct keys, and simply had one transaction slower to commit than its neighbors.
- **dc30111** (2026-08-02) — “*nearest_key_boundary: guarantee a capacity-fitting split when one exists.*” The boundary search used to combine two “don’t tear a key’s version chain” and “fit within capacity” constraints greedily and could miss a valid boundary that satisfied both (example from the commit: 31 records, capacity 16, the nearest non-tearing boundaries sit at 14 and 17 — both over capacity — even though a capacity-fitting boundary exists further out). The fix searches in tiers, in order of preference: (1) no tear and fits, (2) fits but may tear, (3) no tear but may overflow, (4) fall back to the raw midpoint (`smo.rs:76–117`).

4.3 Merge: choosing what to combine

- **dcb5abe** (2026-08-02) — “*merge(): pick the emptier of two adjacent siblings, not always the right one.*” The candidate search now compares available room on both sides instead of defaulting to the right sibling.
- **4c01360** (2026-08-02) — “*merge(): drop the pointless ts_start ordering when combining two leaves.*” A `ts_start`-based re-sort risked reordering a live record ahead of a dead predecessor with a larger `ts_start`, corrupting the backward-scan (`rfind`) physical-order invariant relied on everywhere in §2.
- **0659d34** (2026-08-02) — a redundant `borrow_read().deref()` removed; a candidate-retry-on-failure idea was tried and reverted after it was found to spin without backoff and starve threads under contention.
- **4bc78de** (2026-07-28) — “*Fix GC block reuse in merges; Now correctly calculates the live number.*” Merging two leaves based only on `active_count` could overflow `NUM_RECORDS`, because `record_survives_gc` (§2) also keeps GC-protected dead records that an active-count threshold alone does not see. Fixed by an explicit `leaf_merge_would_overflow` pre-check (`smo.rs:779–820`) that falls through to a `KeySplit` instead when the true survivor count would not fit.

4.4 A week of hardening (2026-07-31 – 08-02)

The commits above were the tail end of a dense, one-week debugging arc chasing two real TPC-C crashes: 7432525 hardened `InternalPage` overflow checks without yet fixing the crash; 5c54658 fixed both crashes via “GC-gated survivor protection” (a correct `record_survives_gc` distinguishing a genuinely-committed delete from one whose abort-reversal still needs the predecessor physically intact) and corrected `bulk_push` accounting (previously assuming every bulk-pushed record was active); 77a5689 reworked split/merge for a “weakened fresh node rule”; and fc8806b reverted `on_overflow_node/on_underflow_node/split_root/merge_root` back to `Result + continue-on-success` now that the underlying bug was actually fixed, undoing a defensive workaround that had been layered on top while the real cause was still unknown.

5 The self-overwrite optimization

This is the trick at the heart of the user’s original question: *when a single, still-uncommitted transaction updates the same key several times, overwrite the previous update in place instead of appending a new multiversion entry.*

5.1 The problem

Recall from §2 that every write mints one more physical `RecordPoint` tuple, and that an *uncommitted* tuple can be neither discarded (the transaction might still commit) nor coalesced with anything (nothing has superseded it yet, since nothing else can even see it). The old `DbTransaction::update` path was literally “delete old, push new” on every call — so if one open transaction wrote the same key N times before committing, it left N physical, uncommitted, and therefore un-reclaimable tuples for that one key in the page.

If N grows large enough relative to a page’s capacity, structural maintenance (§4) is put in an impossible position: `split()`’s `nearest_key_boundary` needs a key boundary to cut at, but a page dominated by one key’s own version chain has none — the routine’s own documentation (`smo.rs:44–50`) states it falls back to tearing the chain across two disjoint-fence siblings *only* when the page is a single key’s entire chain with no boundary at all, silently orphaning the torn-off half from any fence-routed lookup. Symmetrically, `merge()`’s combine-into-one-leaf path has to worry about overflowing `NUM_RECORDS` on exactly the same accounting (§4). A concrete real-world trigger: TPC-C’s New-Order transaction pricing two order-lines for the same item lands two updates on the very same `Stock` row inside one transaction.

5.2 The fix

Current code, `src/mv_db/transaction.rs:251–311` (`DbTransaction::update`, lightly abridged):

```
match leaf_page.as_records_mut().iter_mut()
    .rfind(|r| r.key() == key && !r.version.insertion_stamp().is_invalid())
{
    Some(record) =>
        if tree.is_visible_stamp(self.worker_id, self.ts_start,
                                record.version.insertion_stamp())
        {
            let stamp = TxStamp::new(self.worker_id, self.ts_start);
            tree.wal_log_write(stamp, |_| CRUDOperation::Update(key, payload.clone()));

            // Self-overwrite fast path: this record was already written
            // by *this* transaction -- nobody else can possibly be
            // relying on its current content, so mutate it in place.
            if record.version.insertion_stamp() == stamp {
                record.set_payload(payload);
            }
        }
}
```

```

        return CRUDOperationResult::Updated(stamp.ts_start());
    }

    // Otherwise: normal versioned update (delete old, push new).
    if !record.version.delete(stamp) { /* conflict */ }
    let len = leaf_page.len();
    leaf_page.push_uncommitted(
        RecordPoint::new(key, VersionInfo::new(stamp), payload), len);
    leaf_page.commit_delta(0, 1);
    self.written.borrow_mut().push((table, key));
    CRUDOperationResult::Updated(stamp.ts_start())
} else { CRUDOperationResult::Conflict },
None => CRUDOperationResult::ZeroAffected(KeyDoesNotExist),
}

```

Detection is a single equality check, not a heuristic: a `TxStamp` packs (`worker\_id`, `ts\_start`) and is constant for the *whole* transaction (one stamp per transaction, not one per write — §2). So “was the newest live version of this key written by me, in this very transaction” reduces to `record.version.insertion_stamp()==stamp`.

Safety follows directly from visibility: `is_visible_stamp` only returns true for the same worker or a stamp already present in the commit log (§9); neither applies to any other transaction while this one is still open. There is therefore no possible reader anywhere relying on the current content of this specific uncommitted tuple — unlike a normal update, there is no already-committed predecessor to protect, because nothing has committed yet. If the found record instead belongs to another transaction, or is an older, already-committed version, the code falls through to the ordinary delete-then-push path unchanged.

5.3 The chronological story

Two separate mechanisms are easy to conflate here; only the second is the trick this section is about.

An older, unrelated mechanism: GC-driven update-in-place (2025–2026). `bea1117` (2025-06-05) added an *update-in-place* optimization to the single-operation (non-transactional) CRUD dispatch path (`src/mv_query/dispatch.rs`): if a record’s insert version predates every live snapshot — i.e., GC has already determined nobody can still need the old *committed* value — its payload is overwritten in place rather than versioned again. `16d1402`, `7b1c466`, `fad58a6`, `7623f41` (2026-02-02 to 02-04) incrementally wired and debugged the feature flag controlling this. It is gated off whenever a WAL is attached (the fast path cannot be represented as a single WAL record) and remains in the codebase today, but it is a GC/version-bloat heuristic for single-op writes, unrelated to the transactional self-overwrite trick.

The actual fix (August 2026).

1. `160186e` (2026-08-01) — while adding concurrent TPC-C/YCSB/TPC-H stress tests, discovered that `DbTransaction::abort()` only reverted the *newest* of several same-key writes, leaving older self-written versions permanently visible to that worker. Fixed `abort_write/apply_invalidate` to unwind the whole chain one physical version at a time. The commit message first names, but does not yet fix, the risk of a single hot key’s version chain being torn across a same-key split.
2. `85ac1b0` (2026-08-01) — **the fix itself**, as shown above. The commit message frames it exactly via the TPC-C Stock-row scenario. It also documents a *reverted* companion idea — lowering the leaf overflow trigger to buy compaction more headroom — abandoned because

it caused a hang under concurrent stress (a separate, pre-existing race left uninvestigated as out of scope).

3. [fbedd74](#) (2026-08-01) — “*Add minimal, ignored repro for the hot-key version-tearing limitation.*” A two-thread, `\#[ignore]`d repro isolating a *different* contributor to the same class of pressure: a long-open snapshot keeping an old committed version of a hot key GC-protected while other transactions keep committing new versions of it — explicitly no same-transaction repeats are involved. The test’s own documentation states that the self-overwrite fast path “cannot help here and doesn’t change this repro’s outcome — it fixes a different, narrower contributor to the same class of page pressure, not this one.”
4. [2a2607e](#) (2026-08-02) — tracing the repro further, the real cause turned out to be the split off-by-one from §4, not hot-key tearing at all: the failing leaf held several distinct keys. The repro file was renamed from `known_issue_hot_key_version-tearing_repro.rs` to `leaf_split_off_by_one_regression_tests.rs` to reflect the real cause.

5.4 What remains open

The self-overwrite fix removes exactly one well-understood source of pile-up: N writes to one key by one transaction now cost one physical tuple, not N . It does not, by itself, solve hot-key leaf-split tearing in general: a page whose survivors are genuinely *all* versions of one key (e.g. many concurrent, non-self, committed updates to one hot key, protected by a long-open reader snapshot) still has no valid key boundary for a `KeySplit`. This is documented as an accepted, low-probability residual limitation in `tests/bench_tpcc_stress_tests.rs:34-51`, with a small numeric tolerance in that test’s invariants rather than exact equality.

6 Garbage collection: reclaiming dead blocks without contention

6.1 From a global mutex to sharded, lock-free reclaim

- [acdc7a8](#) (2024-02-14) — “*Started GC.*” First implementation: a global `Mutex<RbTree<Snapshot>>` of active transactions and a global `Mutex<LinkedList<(*mut Version, BlockRef)>>` of dead pages; allocation pops the front of the dead-page list if it predates the oldest live snapshot. The commit message already names the problem that would dominate the next several months: “*need to consider cases where merge might become impossible due to re-used pages...locks must be left out, or possible deadlocks appear!*”
- [2e3ad21](#) (2024-02-22) — allocation switches to `try_lock()` on both structures (bail out rather than block), avoiding a blocking dependency between allocation and the block-latching path.
- [95e149f](#) (2024-02-29) — “*Exchange GC ActiveTx with Concurrent B+Tree instead.*” The active-transaction index moves from a mutex-guarded red-black tree to the project’s own OLC-locked concurrent B+-tree (`cc_bplustree`), dispatched via `PeekMin/Insert` operations instead of a globally exclusive lock. (This design is itself later superseded — see below.)
- [2bae83c](#), [e4fd950](#) (2024-03-04, 03-07) — two rounds of memory leak fixes in the reuse/dead-page path.

Current end state (`src/mv_gc/block_tracer.rs`, `src/mv_gc/tracker_handle.rs`): liveness is driven directly off `TxContext::live_min_snapshot()` (§8) rather than a separate active-tx tree, and reclaim is driven off a *worker-sharded* `SkipMap` rather than a single list or B-tree — both changes are part of the sharding story in §8.

6.2 Two more races found by ThreadSanitizer, and how each was closed structurally

- d0e0977 (2026-07-21) — “*Stabilize GC and liveness in nodes, now re-added the obsolete/retired mark explicitly.*” A separate `retired:AtomicBool` field is folded into the `RETIRED_FLAG_VERSION` bits of the `SmartCell`’s own `cell_version` word (§3). Before this, a two-step “mark retired, then let `Drop` unlock later” pattern raced a concurrent reuse’s `clear_retired()` against the dying guard’s deferred `Drop`, which stored a value computed from *lock-acquisition* -time state and could silently revert a freshly reused cell’s `cell_version` — confirmed empirically as hangs/livelocks under GC plus heavy concurrency.
- ecc9900 (2026-07-27) — “*Fix issues GC related and stabilize SMO with racey stamps.*” Two independent bugs:
 1. A `DeadPageKey` collision: the dead-block index was keyed by a bare `Version`, but a merge’s two dying siblings deliberately share one death version (the replacement block’s birth version); `SkipMap::insert` on a duplicate key silently displaces the existing entry, permanently leaking one of the two retired blocks. Fixed by keying on `(Version, block\_address)` instead.
 2. A genuine torn-read race in `VersionInfo`: `insert_stamp/ delete_stamp` were plain, non-atomic `TxStamp` fields mutated by `delete/invalidate` while lock-free OLC readers read the same word with no lock and no re-validation — confirmed via ThreadSanitizer. Fixed by backing both stamps with `AtomicU64` and `Relaxed` load/store (sufficient because the hazard was torn reads, not ordering).

The same commit adds `nearest_key_boundary` (§4) to prevent a merge/split cut from tearing a single key’s version chain across two disjoint-fence siblings.

- fe03d90 (2026-07-28) — “*Remove version_region atomic need; Changed SMO Liveness, now only checks overlap without actual need to obsolete mark.*” The capstone of this whole thread: `InternalPage::version_region` changes from an array of `AtomicVersion` to plain `Version`, and the `mark_version_obsolete` mechanism — an in-place `fetch_or` on an already-published, concurrently lock-free-read slot, itself confirmed via ThreadSanitizer as a genuine race against range-query iteration — is deleted outright. It is replaced by `InternalPage::live_mask()`: slot i is dead *iff* some later slot $j > i$ ’s key interval overlaps slot i ’s — an $O(\text{FAN_OUT}^2)$ pure-integer check that runs only on the SMO cold path, exploiting the invariant that live sibling intervals are always pairwise disjoint and supersession is append-only. No writer ever mutates a slot after publication, so the race does not merely get fenced correctly — it stops being possible.

Recurring pattern. Nearly every “racey”/“stabilize” fix in this section follows the same shape: something was mutated *in place* on an already-published, lock-free-readable slot (an obsolete-mark bit, a retired bit, a delete/invalidate stamp), requiring ever more careful atomic/fence discipline to patch around it — until the design was eventually changed so that nothing is ever mutated after publication at all. This mirrors the fence-removal trajectory of §3: converging from “atomics and fences bolted on to patch races” toward “structure the data so the race is structurally impossible.”

7 Root indexing strategies

A reader must first find “the root as of snapshot S ” before it can traverse anything. `RootIndexType` (`src/mv_root/index_root.rs:28–55`) selects one of four structures for this, all keyed by version (Table 2).

Strategy	Structure	Lookup	Notes
Vanilla	<code>LinkedList<Root<..>></code>	$O(n)$ linear <code>rfind</code>	Simplest baseline.
SK (SkipList)	<code>crossbeam_skiplist::SkipMap</code>	$O(\log n)$ expected, <code>range(..=si).last()</code>	Lock-free concurrent inserts/lookups.
DexaBTree	external OLC-locked <code>BPlusTree</code> , keyed by <code>Snapshot</code>	$O(\log n)$ via <code>PeekMax/Pred/Insert</code> CRUD dispatch	Reuses a full concurrent B+-tree just to index root pointers by version.
FrugalList (default)	lock-free singly-linked list with a probabilistic skip-list “tower” (<code>v_ridgy</code> links, coin-toss $p = 0.5$)	expected sub-linear via tower, falling back to the flat chain	Appends are a single <code>AtomicPtr</code> store (no CAS loop needed, since appends are already serialized by the enclosing <code>SmartCell</code> ’s write latch); much smaller per-node footprint than a full skip-list tower.

Table 2: Root-index strategies and their tradeoffs.

These are *not* partitioned by version range — each strategy is one single, tree-wide structure holding every historical root, chosen once at construction. They optimize a different axis than sharding (§8): the cost of finding the root pointer itself, a potential single hot spot every reader must consult, trading lookup complexity against per-append cost and memory overhead.

8 Sharding as a system-wide pattern

The word “sharding” shows up in this codebase in four genuinely distinct places. None of them is classic horizontal key-range partitioning of one index; each instead partitions a specific hot, shared piece of *bookkeeping state* by worker (or, in one case, splits the index space itself by table).

8.1 Tables as shards: one tree per table

5f92e9f (2026-07-15) — “*Introduce Tables per indexes, not encoded into a single tree.*” Before this commit, multiple logical tables were multiplexed into a single tree. `Database (src/mv_db/database.rs:89-115)` now holds `tables:ArcSwap<TableList<..>>`, a list of `TableEntry\{name,tree:Arc<MVBTSt<..>>\}`: each table gets its own physically separate tree, its own root index, and — importantly for §6 — its own dead-block tracker (`TrackerHandleSt` is genuinely per-table). Concurrent transactions touching different tables therefore no longer contend on one tree’s root/internal-page latches or one dead-block skip list. What *must* stay global for cross-table snapshot-isolated transactions — the clock, commit-log pruning, snapshot liveness — moves into one shared, non-generic `TxContext` introduced in the same commit.

8.2 GC dead-block tracker: own-shard-first, round-robin steal

`src/mv_gc/block_tracer.rs:49-62` — `BlockTrace` holds one `SkipMap` shard per CPU (`shard\_for(worker\_id)=worker\_id\%shards.len()`). Registration of a died page (`register\_died\_page`) only ever inserts into the dying worker’s own shard. Reclaim (`try\_reclaim`) pops from the caller’s own shard first; only if that shard is empty (or not old enough) does it fall back

to scanning other shards, starting from a shared round-robin cursor so no single remote shard is favored repeatedly.

Why: death versions all come from one global clock, so before sharding, concurrent SMOs from every worker inserted into a *single* skip list clustering near its current maximum key — an always-contended tail. Since a reclaimer only needs *a* reclaimable block, not the globally oldest one, trading global pop-order precision for per-worker insert locality is a sound tradeoff. f9fc960 (2026-07-23) is the exact patch that added the “try own shard first” fast path (before it, `try_reclaim` always scanned round-robin from a shared cursor even when the calling worker’s own shard already had a usable entry).

A subtlety worth flagging precisely: `mv_wal/writer.rs`’s documentation uses “shard” language for its `hardened` watermark field, and `block_tracer.rs` cites it as design precedent. In the current code, however, there is exactly *one* `WalWriter` per `Database` with a single `hardened: Arc<AtomicVersion>` — no `Vec<WalWriter>` or multi-shard aggregation actually exists today. The “shard” terminology there is vestigial documentation carried over from an earlier design, not a live mechanism.

8.3 TxContext: cache-padded, per-worker atomic slots

`src/mv_sync/tx_context.rs:44-163` holds three parallel vectors, one slot per `WorkerId`: `live_tx` (each worker’s currently published outermost snapshot), `live_tx_depth` (reentrancy depth, so only the outermost `begin_snapshot` call touches `live_tx`), and `in_flight_bound` (a conservative lower bound published while a worker is mid-registration, i.e. has drawn a `ts\start` but not yet inserted into `live\_tx`). Each worker only ever writes its own slot; `free_block/live_min_snapshot` scan all slots with `Acquire` loads, $O(\text{max_workers})$, with no cross-slot synchronization required.

This design directly replaces two earlier shared, globally-contended structures, and the commit history records the profiling numbers that motivated the replacement:

- `live_tx` replaced a shared `SkipMap<Snapshot, AtomicUsize>` (`mv\_gc::query\_tracer`) that, per an in-code comment, was measured eating “roughly a quarter to half of all CPU cycles” on a saturated TPC-C run.
- `in_flight_bound` replaced a single global `registrations\_in\_flight: AtomicUsize` that, instrumented under a 16-thread update-heavy YCSB workload, rejected 73.8% of `free_block` attempts (versus 5.2% at 2 threads).

eac6e40 (2026-08-02) then wrapped all three `Vec<Atomic*>`s in `crossbeam_utils::CachePadded`: previously, up to 8 adjacent workers’ slots could share one 64-byte cache line — false sharing that directly contradicted those fields’ own design-intent comments (“no shared cache line with any other worker’s slot”). This is a pure layout change (`CachePadded<T>` derefs transparently) targeted at 64-core/128-thread scale, where cross-CCD coherence traffic is expensive.

8.4 Thread-local worker identity and snapshot cache

`src/mv_sync/worker.rs` hands out a `WorkerId` once per (tree, thread), cached in a thread-local keyed by a process-wide `tree\_uid`. A second thread-local, `SNAPSHOT\_CACHE`, gives each worker its own `SnapshotCache` — the OSIC paper’s “thread-local snapshot cache” (§9) — rather than a `Vec` on the tree indexed by `WorkerId`: using actual thread-local storage means no thread ever *could* touch another worker’s cache, a stronger guarantee than “happens to always pass its own id.” This is the assignment mechanism that feeds `worker\_id` into both the `TxContext` slots above and the GC block-tracer shards.

9 OSIC integration: instant commit, lazily hardened

9.1 What OSIC is

Ordered Snapshot Instant Commit [3] decouples *visibility* from *durability*. Every transaction draws two timestamps from one global, monotonically increasing atomic counter (the Global Logical Clock, GLC): `ts_start` at begin, `ts_commit` at commit. Each of a fixed set of long-lived worker threads keeps its own small, append-only commit log — an ascending list of its own committed timestamps. A write becomes visible to another worker’s snapshot the moment its writer’s commit timestamp is appended to that worker’s log: an $O(1)$, purely in-memory step — there is no waiting for the WAL to reach disk. Durability catches up asynchronously and in batches (*group commit*): writes and commit markers are appended to a WAL, and a background thread periodically flushes/fsyncs everything accumulated so far in one I/O, publishing a “hardened” watermark that callers can poll explicitly if they need a durability guarantee.

9.2 How cMVBT’s commit path works today

- `src/mv_sync/clock.rs` — `GlobalClock`: a single `AtomicVersion`, `fetch\_add(1, SeqCst)` mints both `ts\_start` and `ts\_commit`. There is no separate “publish” step: a drawn timestamp is immediately comparable.
- `src/mv_sync/commit_log.rs` — `CommitLog`, one per worker, a `Mutex<Vec<Version>>`. `commit()` draws `ts\_commit` then pushes it under the lock; this ordering (the mutex serializes the append against any concurrent reader) is exactly what prevents a reader from observing a reserved-but-not-yet-inserted commit stamp. `commit\_pruned` additionally trims the log once it exceeds `max\_workers` entries still needed as *some* live snapshot’s lower commit bound (LCB).
- `src/mv_sync/visibility.rs` — `is_visible`: a direct port of the paper’s core visibility check. An invalid (aborted) stamp is never visible; same-worker writes are visible if `stamp.ts\_start() <= reader\_ts\_start`; otherwise visibility is `LCB(stamp.worker\_id, reader\_ts\_start) > stamp.ts\_start()`, computed via each thread’s own `SnapshotCache` memo.
- `src/mv_wal/writer.rs` — `WalWriter`: a lock-free MPSC channel to one background flush thread. `log\_with\_stamp/log\_commit` just encode and enqueue, returning immediately (fire-and-forget, no per-op fsync wait). `flush\_loop` sleeps a short `GROUP\_COMMIT\_LINGER` (200 μ s) after the first message, then drains everything else non-blockingly, batches it into one `write\_all + sync\_data()`, and advances the `hardened` watermark to the maximum `ts\_start` of any *Commit* message in that batch — never based on a bare write.
- `src/mv_wal/record.rs` — the wire format distinguishes `WalEntry::Write` from `WalEntry::Commit`: `{stamp, ts\_commit}`: a commit marker is a separate append-only entry because a write is logged optimistically *before* its transaction’s fate is known.
- `src/mv_wal/recovery.rs` — replay keeps only `Writes` with a matching `Commit` marker (commit-gated replay), and replays sorted by `(ts\_commit, fileorder)`, not by `ts\_start`, since commit order can differ from log order across concurrent workers.
- `src/mv_db/transaction.rs` — `DbTransaction::commit`: appends to the committing worker’s `CommitLog` (making every write from every table this transaction touched visible everywhere, instantly), then logs exactly one fire-and-forget WAL commit marker. *This is the whole “instant commit”*: one in-memory append, with nothing on the write/commit path waiting on disk I/O.

Contrast with naive MVCC. A naive “lock, fsync, then publish” design pays a full flush round-trip, serially, on every commit. Here, visibility and durability are fully decoupled: visibility costs one atomic append; durability is a separate, batched, background concern, exposed only via an explicit `wait\_wal\_hardened()/wal\_hardened\_version()` API for callers who specifically need a durability checkpoint.

9.3 Highlights and differences versus the paper

Paper concept (LeanStore/OSIC)			cMVBT realization		Notable difference
Global (GLC)	Logical	Clock	GlobalClock, fetch_add	single	Direct port; simplified from an earlier ad hoc thread-indexed watermark array (see below).
Per-worker LCB	commit log	+	CommitLog	+	Direct port, with an added commit_pruned mode that trims the log even when block-reclaim GC is disabled, since an OSIC-style log otherwise grows unboundedly during long GC-off historic-query runs.
Early Lock Release of the paper)	(§3.4)		Removed		The paper’s pessimistic per-commit durability cap (blocking dependents on one’s own flush) was replaced by a simpler per-writer three-state watermark advanced per batch, with no caller blocking by default — judged not worth the cost of a flush round-trip on every single operation.
Version storage			Versions live directly inside the multiversion B-tree’s own pages (§2), not a separate version chain/undo log as in LeanStore’s single-version core		OSIC’s visibility/commit machinery is layered on top of an already-multiversion structure, so “old version still visible” is a property of the tree itself, not of a side chain that the commit protocol must also manage.
Group commit			GROUP_COMMIT_LINGER batching in flush_loop		Direct port of the batching idea, tuned empirically (200 μs) for this workload mix.

Table 3: OSIC as described in [3] versus its realization here.

9.4 Evolution: two eras

Pre-OSIC era (Dec 2025 – Jan 2026). Before the paper’s design was adopted wholesale, the system used a single global commit-version counter with an explicit publish step, plus a hand-rolled sharded thread-indexed “committed” watermark array — in effect, an imperfect reinvention of the CommitLog/GLC pair:

- 17b8f5f (2025-12-17) — “*Changed commit from CAS to fetch_add via SeqCst.*” Replaced a `compare\_exchange\_weak` retry loop with a bare `fetch\_add`, since commit numbering no longer needed a conditional check.

- 0358c1c/194f973/87e2523 (2025-12-20/21) — “*Fix SI for update operations; divided into two atomic sequential operations, as described in the paper.*” The old update path interleaved commit-number allocation with the leaf mutation itself, using an admitted stopgap: a `delete(key, committed\_version+1)` “fixed visibility window” fudge factor (the commit’s own message calls this out: “*For now, I use a fixed visibility window*”). One day later, this was replaced with the discipline the current code still follows: publish the new version first, *then* mark the old one deleted — closing the gap where an old version could go invisible before its replacement was guaranteed visible.
- 047d094, 20396d7, efd7a6, 408becb, 4981858, 67606e4/11effc9/7d43e8e, 836a1db (Jan 2026) — a sequence of patches to a thread-id-indexed watermark array, repeatedly fixing edge cases around threads changing roles (an OLTP worker becoming a reader mid-run) or exiting, and a query tracer needing its own clock thread-id. The commit messages read as a slow discovery, by trial and error, of exactly the problem OSIC’s per-worker commit-log design solves cleanly.

OSIC era (from 714a82f, 2026-07-10). A large commit, “*OSIC/WAL and TPC-C/OLAP Benchmark,*” wholesale replaces the ad hoc clock/ watermark scheme with the paper’s actual design: `GlobalClock`, per-worker `CommitLog`, `WorkerRegistry` + `SnapshotCache`, and the entire `mv\_wal` module, alongside new TPC-C/OLAP benchmark harnesses.

Post-OSIC refinement (Jul 13 – 28, 2026).

- 3e8b837 — “*Add group commit linger to collect and fsync logs more frequently.*” Added the 200 μ s linger delay described above, trading a little latency for far higher throughput per fsync.
- ab10660 — “*Make WAL worker non-blocking for callers and batch hardening.*” Removed the paper’s Early Lock Release machinery (a `hardened` field plus `mark\_pending/mark\_resolved` letting a committing worker cap the global durability watermark at its own `ts\_commit-1` until its WAL entry flushed, blocking dependents) in favor of a per-writer three-state watermark advanced per batch with no default blocking.
- 73cc4a5/a5e6b7a — “*Add commit log truncate setting, turned on by default (gc = off + truncate_commit_log = on).*” Added `allow\_historic\_query(enabled)` (superseding an earlier, confusingly-named `truncate\_commit\_log(enabled)`), coupling and inverting the tradeoff explicitly: enabling historic queries disables both GC and commit-log truncation, since a run that needs to query old snapshots cannot prune the very log that lets it find them, while a normal OLTP run should prune aggressively to bound memory (documented cost without pruning: page faults from `Vec` growth showing up as $\sim 2.4\%$ of commit’s own profiled samples).
- ecc9900, fe03d90 — the same GC/SMO stabilization fixes described in §6, adjacent to (and interacting with) the OSIC-visible commit stream.

9.5 Known, documented limitations

- **Unbounded CommitLog growth** under long GC-off historic-query runs: page faults from `Vec` growth are directly attributed to commit’s own call stack. A chunked/segmented log was considered but explicitly deferred (“a real rewrite of `prune/lcb\_index`’s cross-block search”).
- `MAX_WORKERS_CAP=256` — a fixed-size inline array backing `SnapshotCache`, generous but still a hard ceiling.

- **Abort ordering:** `DbTransaction::abort/Drop` must revert writes in strict LIFO order, because reverting exposes each key individually to concurrent readers mid-abort — a documented but delicate invariant, not a generically solved problem.
- **Self-overwrite leaves one physical entry, not zero:** writing the same key N times in one transaction still leaves one extra physical entry versus never writing it at all — enough, under stress, to occasionally contribute to a same-key leaf split (§5).
- The scope of `ecc9900`’s torn-read fix is explicit in its own documentation: it closes one specific torn-read hazard, not every possible concurrent-mutation hazard on the lock-free OLC read path in general.

10 Timeline

Table 4 indexes the commits discussed in this document chronologically, for cross-reference against `git log`.

Commit	Date	Theme
acdc7a8	2024-02-14	GC started (§6)
95e149f	2024-02-29	GC active-tx index moved to a concurrent B+-tree (§6)
d565b2d–5c26a02	2024-01 – 2024-12	ORWC implemented, iterated, deprecated, purged (§3)
48c4ded, a7bb29c, a518eed	2024-05 – 2025-04	Fence removal / ordering tightening (§3)
0cd4fee, fa82c8c, d97de21	2024-03	Early DBTracker / snapshot bookkeeping (§8)
bea1117, 16d1402–7623121	2025-06 – 2026-02	GC-driven update-in-place (unrelated mechanism, §5)
17b8f5f, 0358c1c, 194f973/87e2523	2025-12	Pre-OSIC commit/visibility fixes (§9)
047d094–836a102	2026-01	Pre-OSIC thread-watermark patching (§9)
34555e2	2026-06-20	Overflow-trigger tuning (§4)
2a2be93, 40b00fd	2026-06	Further ordering relaxation (§3)
714a82f	2026-07-10	OSIC/WAL adopted wholesale (§9)
3e8b837, ab10660, 73cc4a5/a5e6b7a	2026-07-13/15	Group commit, ELR removal, log truncation (§9)
5f92e9f	2026-07-15	Tables-as-shards (§8)
f9fc960	2026-07-23	GC own-shard-first + steal (§8)
d0e0977	2026-07-21	Retired-bit race fix (§6)
45c7c21/fd92132	2026-07-21	Conservative child latch removed (§3)
7f2bd09	2026-05-21	Cache-line isolation for node tag (§3)
77a5689, 5c54658, fc8806b, 7432525	2026-07-31	TPC-C crash hardening week begins (§4)
ecc9900	2026-07-27	Racey-stamp + dead-page-key collision fixes (§6)
fe03d90	2026-07-28	Obsolete-mark removed; overlap-based liveness (§6)

Commit	Date	Theme
4bc78de	2026-07-28	Merge-overflow accounting fix (§4)
160186e	2026-08-01	Repeated-key abort-revert fix (§5)
85ac1b0	2026-08-01	Self-overwrite fast path (§5)
fbedd74	2026-08-01	Hot-key repro added (§5)
2a2607e	2026-08-02	Split off-by-one found and fixed (§4, §5)
dc30111, 4c01360, dcb5abe, 0659d34	2026-08-02	Merge/boundary-search fixes (§4)
eac6e40	2026-08-02	Cache-padded per-worker atomics (§8)
b29d347	2026-08-02	FAN_OUT tuned for page alignment (§4)

Table 4: Chronological index of commits discussed in this document.

11 Conclusion

Read end to end, the history of this codebase tells a consistent story at every layer: an initial design gets something working with coarse-grained locks, mutexes, or bolted-on atomics/fences; profiling or ThreadSanitizer then reveals exactly where that coarseness costs correctness (a race) or throughput (contention); and the fix is, more often than not, not “add a lock” but “restructure the data so the hazard cannot occur” — append-only pages instead of in-place edits, interval-overlap-derived liveness instead of a mutable obsolete bit, per-worker sharded bookkeeping instead of one shared structure, instant in-memory commit instead of commit-time fsync. The self-overwrite trick in §5 is a small, local instance of exactly this pattern: rather than adding machinery to garbage-collect or specially-handle a hot key’s pile of uncommitted versions, the fix removes the pile-up from ever occurring in the first place.

References

- [1] Becker, B., Gschwind, S., Ohler, T., Seeger, B., Widmayer, P. *An asymptotically optimal multiversion B-tree*. The VLDB Journal, 5(4):264–275, 1996.
- [2] Tonta, A., Seeger, B., Soisalon-Soinen, E. *Multiversion Concurrency Control for Multiversion B-Trees*. arXiv preprint arXiv:2606.09133, 2026.
- [3] Alhomssi, A., Leis, V. *Scalable and Robust Snapshot Isolation for High-Performance Storage Engines*. Proceedings of the VLDB Endowment, 16(6):1426–1438, 2023. doi:10.14778/3583140.3583157.